inmind.academy
by inmind.ai

Multimodal Pipelines + Evaluation

Voice, Vision, and Measuring What Matters

SPRING 2026

inmind.academy
by inmind.ai

Today's Roadmap

Two halves: first, make your agents impressive with voice and vision. Second, learn to measure everything properly.

- **Part 1:** Speech Technology Landscape
- **Part 2:** Whisper Deep Dive
- **Part 3:** Text-to-Speech Options
- **Part 4:** Full Voice Pipeline
- **Part 5:** Image Understanding (Applied)
- **Part 6:** Multimodal Agents
- **Part 7:** Why Evaluation Matters
- **Part 8:** Retrieval Evaluation Metrics
- **Part 9:** Generation Evaluation
- **Part 10:** RAGAS Framework
- **Part 11:** LLM-as-Judge
- **Part 12:** Observability and Monitoring

Where We Left Off

In Session 10, you built multi-agent systems with supervisors, specialist agents, MCP for external services, and guardrails for safety. You have a working multi-agent RAG system.

Now we add two capabilities: multimodal I/O (so users can speak to your agent and see images) and evaluation (so you can prove your system works). Both are critical for going from prototype to production.

Part 1: Speech Technology Landscape

Before we write code, let's understand how speech technology works, what the options are, and where the field stands in 2025.

How Automatic Speech Recognition Works

Speech recognition has evolved through three generations:

- **Traditional pipeline (pre-2015):** Audio -> Feature extraction (MFCCs) -> Acoustic model (HMM/GMM) -> Language model (n-gram) -> Text. Each component trained separately. Errors cascade. One model per language.
- **End-to-end deep learning (2015-2022):** Audio -> Single neural network (encoder-decoder) -> Text. Models like DeepSpeech, Wav2Vec2, Conformer replaced the pipeline. Better accuracy, but still one model per language.
- **Foundation models (2022-present):** Audio -> Large pretrained model (Whisper, USM) -> Text. Trained on massive multilingual datasets. One model handles 99+ languages. Zero-shot on unseen languages. This is what we use today.

inmind.academy
by inmind.ai

Speech-to-Text Models in 2025

Model	By	Languages	Local/API	Open Source	Key Strength
Whisper large-v3	OpenAI	99+	Both	Yes	Best open-source, multilingual
Whisper large-v3-turbo	OpenAI	99+	Both	Yes	8x faster, near-same accuracy
Faster-Whisper	Community	99+	Local	Yes	CTranslate2 backend, 4x faster
Deepgram Nova-2	Deepgram	36	API	No	Fastest API, real-time streaming
Google Speech v2	Google	125+	API	No	Best language coverage
Azure Speech	Microsoft	100+	API	No	Enterprise features, custom models
Parakeet	NVIDIA	English	Local	Yes	State-of-the-art English accuracy
MMS	Meta	1,100+	Local	Yes	Most languages supported

For this course: Whisper (or faster-whisper) locally. Free, handles Arabic/French/English, runs on any laptop.

Whisper: How It Works Inside

Whisper is an encoder-decoder transformer trained on 680,000 hours of multilingual audio (OpenAI, Sept 2022).

- 1. Audio -> 80-channel log-mel spectrogram
- 2. Spectrogram split into 30-second chunks
- 3. Transformer encoder processes each chunk
- 4. Decoder generates text tokens autoregressively
- 5. Special tokens control behavior

- **Transcription:** audio in language X -> text in X
- **Translation:** audio in ANY language -> text in English
- **Language detection:** identify which language is spoken
- **Timestamps:** word-level or segment-level timing
- **VAD:** detect speech vs silence

Whisper Model Sizes: Speed vs Accuracy

Model	Params	VRAM	Rel. Speed	English WER	Multi WER	Best For
tiny	39M	~1GB	32x	~7.7%	~12.0%	Quick experiments, edge
base	74M	~1GB	16x	~5.8%	~10.0%	Fast prototyping
small	244M	~2GB	6x	~4.3%	~7.6%	Good balance, any laptop
medium	769M	~5GB	2x	~3.5%	~6.5%	High accuracy
large-v3	1.55B	~10GB	1x	~2.7%	~4.8%	Best accuracy
large-v3-turbo	809M	~6GB	8x	~2.9%	~5.1%	Best speed/accuracy ratio

WER = Word Error Rate (lower is better). A WER of 5% means roughly 1 in 20 words is wrong.

Recommendation: "small" for development. "large-v3-turbo" for production. "large-v3" only when you need maximum accuracy.

Whisper and Arabic: What to Expect

- **Dialectal variation:** MSA is formal, but people speak in dialects (Lebanese, Egyptian, Gulf, Moroccan). These are very different. Whisper handles MSA well but struggles more with dialects.
- **Code-switching:** In Lebanon, people mix Arabic, French, English in one sentence. Whisper can transcribe this but may switch language tags mid-sentence.
- **Diacritics:** Arabic can be written with or without tashkeel. Whisper outputs without diacritics by default, which can create ambiguity.
- **Performance:** Whisper large-v3: ~8-12% WER on MSA, ~15-25% on dialects. Compare to ~2-4% on English. Arabic accuracy improved significantly from v1 to v3.
- **Practical tip:** For Lebanese Arabic: faster-whisper + large-v3-turbo gives best results. For MSA (news, formal): even the small model works well.

inmind.academy
by inmind.ai

Whisper in Practice: Code Examples

```python
# Option 1: OpenAI Whisper
import whisper
model = whisper.load_model("small")
result = model.transcribe("audio.mp3")
print(result["text"])
print(f"Language: {result['language']}")
```

```python
# Option 2: Faster-Whisper (4x faster)
from faster_whisper import WhisperModel
model = WhisperModel("large-v3-turbo",
    device="cpu", compute_type="int8")
segs, info = model.transcribe("audio.mp3")
for seg in segs:
    print(f"[{seg.start:.1f}s] {seg.text}")
```

```python
# Option 3: OpenAI API (cloud, no local GPU needed)
from openai import OpenAI
client = OpenAI()
with open("audio.mp3", "rb") as f:
    transcript = client.audio.transcriptions.create(model="whisper-1", file=f, language="en")
print(transcript.text)
```

Part 2: Text-to-Speech (TTS)

Making your agents speak. The TTS landscape, quality comparisons, and practical integration.

How Text-to-Speech Works

- **Concatenative TTS (1990s-2010s):** Record thousands of short audio segments. Stitch together at runtime. Sounds robotic. Old GPS and phone menus.
- **Statistical parametric (2010s):** Model generates speech parameters, vocoder converts to audio. More flexible but still synthetic. Early Google Translate voices.
- **Neural TTS (2017-present):** End-to-end deep learning. Neural network generates spectrogram, neural vocoder (WaveNet, HiFi-GAN) converts to audio. Sounds nearly human. Modern Google/Amazon/Azure TTS.
- **Zero-shot voice cloning (2023-present):** Given 3-10 seconds of someone's voice, generate speech in that voice. XTTS v2, Bark, OpenAI TTS. Incredible quality but raises deepfake concerns.

inmind.academy
by inmind.ai

TTS Options in 2026

Option	Type	Quality	Speed	Languages	Cost	Notes
gTTS	API (Google)	Basic	Fast	50+	Free	Simple, pip install gtts
pyttsx3	Local (OS)	Low	Instant	Varies	Free	Offline, OS speech engine
OpenAI TTS	API	Excellent	Fast	57	$15/1M chars	Best quality/simplicity
Google Cloud TTS	API	Excellent	Fast	50+	$4-16/1M chars	Most voice options (400+)
Azure Neural TTS	API	Excellent	Fast	140+	$15/1M chars	Best language coverage
ElevenLabs	API	Premium	Fast	29	$5-330/mo	Best voice cloning
Coqui XTTS v2	Local	Very good	Slow	17	Free	Open-source voice cloning
Piper	Local	Good	Very fast	30+	Free	Optimized for RPi, lightweight

Quick prototyping: gTTS (free). Quality demos: OpenAI TTS. Local/offline: Piper (fast) or Coqui XTTS (cloning).

inmind.academy
by inmind.ai

TTS in Practice: Code Examples

```python
# gTTS (simplest, free)
from gtts import gTTS
tts = gTTS(text="Hello from TechNova!",
           lang="en")
tts.save("response.mp3")

# Arabic
tts_ar = gTTS(text="مرحبا", lang="ar")
tts_ar.save("response_ar.mp3")
```

```python
# OpenAI TTS (high quality)
from openai import OpenAI
client = OpenAI()
response = client.audio.speech.create(
    model="tts-1",
    voice="nova",
    input="Welcome to TechNova!"
)
response.stream_to_file("out.mp3")
```

```python
# Piper (local, fast, lightweight)
# Download voice model from: https://github.com/rhasspy/piper
import subprocess
subprocess.run(["piper", "--model", "en_US-lessac-medium.onnx",
    "--output_file", "response.wav"],
    input="Welcome to TechNova support!", text=True)
```

What Makes TTS Sound Good or Bad?

- **Naturalness:** Does it sound human? Neural TTS solved this for short sentences. Longer paragraphs can still sound monotone.
- **Prosody:** Rhythm, stress, intonation. Good TTS varies pitch naturally. Bad TTS reads every word the same way.
- **Latency:** Time from text to audio. Real-time needs < 500ms. gTTS: 1-3s (network). OpenAI TTS: streaming ~100ms first audio. Local models vary.
- **Streaming:** Can you play audio before full response is generated? Critical for conversation. OpenAI and Azure support it. gTTS does not.
- **Voice consistency:** Does the voice stay the same across sentences? Some models vary slightly between calls.
- **Multilingual:** Can the same voice switch between English and Arabic in one sentence? Most struggle. ElevenLabs and Azure handle it best.
- **Cost at scale:** 1,000 conversations/day = millions of characters. At $15/1M chars, that adds up fast.

The Full Voice Pipeline: Architecture

```
[Microphone] -> [STT (Whisper)] -> text -> [Agent/LLM] -> text -> [TTS] -> [Speaker]
                                            |                       |
                                           +-- Conversation History +
```

Component	Target Latency	Typical Latency	Notes
Recording + silence detection	200-500ms	Variable	Wait for speaker to stop
STT (Whisper small, local)	500-2000ms	Depends on length	Longer audio = more time
STT (API)	200-500ms	Network dependent	Faster but needs internet
Agent/LLM processing	500-3000ms	Model + tools dependent	Biggest variable
TTS generation	200-1000ms	Provider dependent	Streaming reduces perceived latency
Total end-to-end	1.5-7 seconds		Acceptable for demo, slow for production

Production: use streaming at every step. Start STT while user speaks. Start TTS while LLM generates. Perceived latency < 2s.

Building the Voice Pipeline: Code

```
import whisper
from gtts import gTTS
import sounddevice as sd
import tempfile, os
from langchain_ollama import ChatOllama
from langchain_core.messages import HumanMessage, SystemMessage, AIMessage

stt_model = whisper.load_model("small")
llm = ChatOllama(model="llama3", temperature=0.7)
history = [SystemMessage(content="You are TechNova support. Keep responses brief.")]

def record_audio(duration=5, sr=16000):
    print("Listening...")
    audio = sd.rec(int(duration * sr), samplerate=sr, channels=1, dtype="float32")
    sd.wait()
    return audio.flatten()

def voice_chat():
    while True:
        audio = record_audio()
        result = stt_model.transcribe(audio, fp16=False)
        user_text = result["text"]
        print(f"You: {user_text}")
        if not user_text.strip(): continue
        history.append(HumanMessage(content=user_text))
        response = llm.invoke(history)
        history.append(AIMessage(content=response.content))
        print(f"Agent: {response.content}")
        tts = gTTS(text=response.content, lang="en")
        with tempfile.NamedTemporaryFile(suffix=".mp3", delete=False) as f:
            tts.save(f.name); os.system(f"afplay {f.name}")
```

Part 3: Image Understanding with VLMs

Applied image Q&A and document understanding.

inmind.academy
by inmind.ai

What VLMs Can Do Today

- **Strong (> 90%):** Describe photos, read printed text (OCR replacement), answer factual questions, identify objects, understand charts, read tables

- **Good (70-90%):** Count objects (> 10), spatial relationships, handwriting, complex diagrams, dense multi-page docs

- **Challenging (< 70%):** Fine-grained visual reasoning, very small text in large images, multi-page reasoning, precise measurements, video sequences

Model	Provider	Local/API	Key Strength
GPT-4o → 5.4	OpenAI	API	Best overall quality
Claude 4.6 Sonnet	Anthropic	API	Best at documents/charts
Gemini 3.0 Flash	Google	API	Free tier, fast
LLaVA	Open source	Ollama	Best local option
Qwen X-VL	Alibaba	Local	Strong document OCR
InternVL2	Open source	Local	Top benchmark scores
Pixtral	Mistral	Both	Native multimodal

Adding Vision to Your Agent

```
from langchain_core.tools import tool
import base64
from langchain_ollama import ChatOllama
from langchain_core.messages import HumanMessage

@tool
def analyze_image(image_path: str, question: str) -> str:
    """Analyze an image and answer a question about it."""
    vlm = ChatOllama(model="llava")
    with open(image_path, "rb") as f:
        img_b64 = base64.b64encode(f.read()).decode()
    response = vlm.invoke([HumanMessage(content=[
        {"type": "text", "text": question},
        {"type": "image_url", "image_url": {"url": f"{img_b64}"}}
    ])])
    return response.content
```

- **Error screenshot:** Customer sends screenshot -> agent reads and diagnoses
- **Receipt processing:** User uploads receipt for return -> agent extracts order info
- **Chart interpretation:** Analyst shares chart -> agent explains trends

inmind.academy
by inmind.ai

Document AI: From PDF to Answers

```
PDF Upload
    |
    v
[Page Extraction] -> images (one per page)
    |
    v
[For each page:]
    |-- Text pages -> [pdfplumber] -> chunks -> embed -> vector DB
    |-- Scanned pages -> [OCR] -> text -> chunks -> embed -> vector DB
    |-- Charts/images -> [VLM description] -> text -> chunks -> embed -> vector DB
    v
[Unified Vector DB with metadata: page_number, extraction_method, content_type]
```

Key insight: different pages in the same PDF may need different processing. A smart pipeline detects page type and routes accordingly. Metadata tracking (extraction_method) helps with debugging.

Part 4: Why Evaluation Matters

Without metrics, you are guessing. The difference between a demo and a production system.

inmind.academy
by inmind.ai

Why Most AI Projects Fail at Evaluation

1. Team builds a RAG chatbot. Works great on 5 test questions they tried by hand.

2. They demo it to stakeholders. Stakeholders love it. "Ship it!"

3. Users start using it. 30% of answers are wrong or hallucinated.

4. Team scrambles to fix issues but has no idea which component is failing (chunking? retrieval? LLM? prompt?).

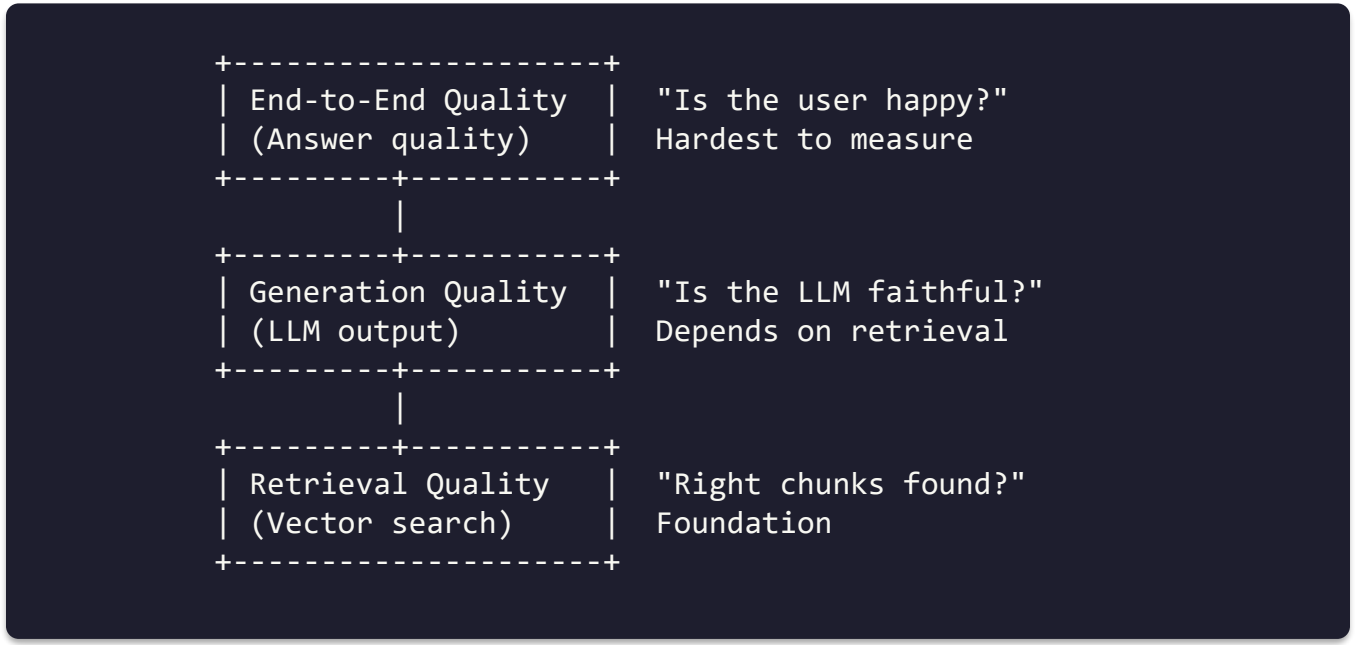
5. They change chunk size. Some things improve, some degrade. No systematic tracking.

6. Months of whack-a-mole debugging. Team loses confidence. Project stalls.

The root cause: no evaluation framework from the start. If they had measured retrieval precision and faithfulness on a test set BEFORE shipping, they would have caught the 30% failure rate.

Evaluation is not a nice-to-have. It is the difference between a prototype that impresses in a demo room and a product that works in the real world.

The RAG Evaluation Pyramid



If retrieval is bad, nothing else matters. Fix retrieval first. Once retrieval is good, evaluate generation. Only after both are solid should you measure end-to-end user satisfaction.

inmind.academy
by inmind.ai

Building an Evaluation Test Set

#	Question	Expected Answer	Source	Chunk
1	Return policy defective?	90 days, full refund + shipping	handbook, Sec 1	chunk_3
2	Pro annual + student?	$329 (470*0.7)	handbook, Sec 2	chunk_8
3	Who is CTO?	Ahmad Khalil, co-founder	handbook, Sec 4	chunk_15

- Minimum 20-30 questions for meaningful evaluation
- Cover all document sections/topics proportionally
- Include easy (answer in one chunk) and hard (spans multiple chunks) questions
- Include adversarial (answer NOT in documents; system should say "I don't know")
- Include different phrasings for the same information
- Include all supported languages if multilingual
- Write expected answers BEFORE running the system (avoid bias)

Part 5: Retrieval Evaluation Metrics

Measuring whether your vector search finds the right documents.

Retrieval Metrics: The Essential Four

- **Precision@K:** Of the K chunks retrieved, how many were relevant? Formula: (relevant in top-K) / K. Example: retrieve 5, 3 relevant -> P@5 = 0.60. Tells you: how much noise in results.
- **Recall@K:** Of ALL relevant chunks in the DB, how many did you retrieve? Formula: (relevant in top-K) / (total relevant). Example: 4 relevant total, retrieved 3 -> R@5 = 0.75. Tells you: are you missing info?
- **MRR (Mean Reciprocal Rank):** How high up is the FIRST relevant result? Formula: 1 / (rank of first relevant). First relevant at pos 1 -> RR=1.0. At pos 3 -> RR=0.33. Tells you: how much scrolling before useful result.
- **NDCG@K:** Like precision but gives more credit for relevant results appearing higher in ranking. Accounts for position. Tells you: overall ranking quality.

Computing Retrieval Metrics: Code

```
import numpy as np

def precision_at_k(retrieved, relevant, k):
    top_k = retrieved[:k]
    return len(set(top_k) & set(relevant)) / k

def recall_at_k(retrieved, relevant, k):
    top_k = retrieved[:k]
    return len(set(top_k) & set(relevant)) / len(relevant) if relevant else 0

def mrr(retrieved, relevant):
    for i, doc_id in enumerate(retrieved):
        if doc_id in relevant:
            return 1.0 / (i + 1)
    return 0.0

# Example
test_cases = [
    {"query": "return policy", "relevant": ["chunk_3", "chunk_4"]},
    {"query": "Pro plan price", "relevant": ["chunk_8"]},
]
for tc in test_cases:
    q_vec = embed_model.embed_query(tc["query"])
    results = collection.query(query_embeddings=[q_vec], n_results=5)
    retrieved = results["ids"][0]
    p = precision_at_k(retrieved, tc["relevant"], 5)
    r = recall_at_k(retrieved, tc["relevant"], 5)
    m = mrr(retrieved, tc["relevant"])
    print(f"P@5={p:.2f}  R@5={r:.2f}  MRR={m:.2f}")
```

inmind.academy
by inmind.ai

Interpreting Retrieval Metrics

Metric	Poor	Acceptable	Good	What to Fix
Precision@5	< 0.40	0.40-0.60	> 0.60	Reduce chunk size, better embedding, metadata filters
Recall@5	< 0.50	0.50-0.70	> 0.70	Increase K, multi-query, check chunk boundaries
MRR	< 0.50	0.50-0.70	> 0.70	Better embedding model, re-ranking

Symptom	Likely Cause	Fix
High precision, low recall	K too small	Increase K, multi-query
Low precision, high recall	Too much noise	Smaller chunks, metadata filters, re-ranking
Low MRR	Embedding quality issue	Better model, query rewriting
Everything low	Fundamental problem	Re-examine chunking + embedding model

Part 6: Generation Evaluation

Measuring whether the LLM answer is correct, faithful, and relevant.

Generation Quality Metrics

- **Faithfulness:** Does the answer ONLY use info from retrieved context? If the LLM adds facts not in context, it is hallucinating. Most critical RAG metric.
- **Answer Relevance:** Does the answer address the user's question? Can be faithful but irrelevant (true info but wrong topic).
- **Correctness:** Is the answer factually correct compared to ground truth? The bottom line.
- **Completeness:** Does it cover all important aspects? Correct but incomplete answers miss key info.
- **Conciseness:** Appropriately long? Not too short (missing), not too long (padded with irrelevant details).

inmind.academy
by inmind.ai

Manual Evaluation: Practical Template

Dimension	Score (1-5)	What to Check
Retrieval	_/5	Were the right chunks found?
Faithfulness	_/5	Grounded in context? Flag hallucinated claims
Relevance	_/5	Does it address the actual question?
Correctness	_/5	Compare to ground truth answer
Completeness	_/5	What key info is missing?

Scoring: 1 = Complete failure. 2 = Mostly wrong. 3 = Partially correct, missing details. 4 = Mostly correct, minor issues. 5 = Perfect or near-perfect.

Time budget: ~1-2 minutes per question with practice. 20-question set takes 20-40 minutes. This is time well spent.

Pro tip: We can have two people score independently, then compare. Disagreements reveal ambiguous questions or unclear criteria.

inmind.academy
by inmind.ai

RAGAS: Automated RAG Evaluation

RAGAS uses LLMs to automatically evaluate RAG pipelines. Instead of manually scoring, a strong LLM judges quality.

Metric	Measures	How It Works
Faithfulness	Answer grounded in context?	LLM extracts claims, checks each against context
Answer Relevancy	Addresses the question?	LLM generates Qs from answer, compares to original
Context Precision	Retrieved chunks relevant?	LLM judges each chunk's relevance
Context Recall	Found everything needed?	Checks if ground truth derivable from context

All scores: 0.0 to 1.0 (higher is better).

Requirements: test set with questions, ground truth, retrieved contexts, generated answers. Uses an evaluator

pip install ragas

Running RAGAS: Code

```
from ragas import evaluate
from ragas.metrics import faithfulness, answer_relevancy, context_precision, context_recall
from datasets import Dataset

eval_data = {
    "question": [
        "Return policy for defective products?",
        "How much is Pro annual?",
        "Who is the CTO?",
    ],
    "answer": [
        "Defective products: 90 days, full refund including shipping.",
        "Pro annual: $470 per user per year.",
        "CTO is Ahmad Khalil, co-founder.",
    ],
    "contexts": [
        ["Defective products can be returned within 90 days for full refund including shipping."],
        ["TechNova Pro Plan: Annual: $470 per user per year (save $118)."],
        ["CTO: Ahmad Khalil (co-founder, previously Lead Architect at CloudScale)"],
    ],
    "ground_truth": [
        "Defective: 90 days, full refund + shipping",
        "$470/user/year",
        "Ahmad Khalil, co-founder",
    ],
}

dataset = Dataset.from_dict(eval_data)
results = evaluate(dataset, metrics=[faithfulness, answer_relevancy, context_precision, context_recall])
print(results)
```

inmind.academy
by inmind.ai

Interpreting RAGAS Results

Metric	Score	Interpretation	Action
Faithfulness	> 0.90	Excellent, sticking to context	Maintain prompt
Faithfulness	0.70-0.90	Some hallucination	Strengthen 'only use context' in prompt
Faithfulness	< 0.70	Serious hallucination	Major prompt rework or different LLM
Context Precision	> 0.80	Relevant chunks retrieved	Good retrieval
Context Precision	< 0.80	Too much noise	Improve chunking, embedding, or add re-ranking
Context Recall	> 0.80	Finding most relevant info	Good coverage
Context Recall	< 0.80	Missing important chunks	Increase K, multi-query, check boundaries

Debugging workflow: 1) Low context precision/recall? Fix retrieval first. 2) Good retrieval but low faithfulness? Fix the prompt. 3) Good faithfulness but low relevance? LLM not understanding the question.

LLM-as-Judge: Using AI to Evaluate AI

```
def evaluate_answer(question, answer, context, ground_truth):
    prompt = f"""Score this answer on:
      1. Correctness (1-5): Factually correct vs ground truth?
      2. Faithfulness (1-5): ONLY uses info from context?
      3. Relevance (1-5): Addresses the question?
      4. Completeness (1-5): Covers all important aspects?

      Question: {question}
      Context: {context}
      Ground truth: {ground_truth}
      Generated answer: {answer}

      Format: Correctness: X/5 - reason
      Faithfulness: X/5 - reason
      Relevance: X/5 - reason
      Completeness: X/5 – reason
      """

    response = judge_llm.invoke([SystemMessage(content="Strict evaluator."),
                                 HumanMessage(content=prompt)])
    return response.content
```

- **Caveat:** Weak model judging itself is unreliable. Use a stronger model as judge. Always validate against human scores on a subset.

Evaluation Best Practices

- **Baseline first:** Evaluate before changing anything. Then change ONE variable, re-evaluate, compare. Without a baseline, you cannot tell if changes helped.
- **Track over time:** Spreadsheet or dashboard. Every change to chunk size, embedding, prompt, LLM: record metrics. This history is invaluable.
- **Adversarial cases:** Questions where the answer is NOT in docs. Tricky phrasings. Unexpected languages. These expose real weaknesses.
- **Separate retrieval from generation:** If end-to-end is bad, where is the problem? Evaluate retrieval independently first.
- **Automate as CI/CD:** Before deploying changes, run test suite automatically. Reject changes that decrease metrics below thresholds.
- **Refresh test sets:** As you fix issues, old cases become too easy. Add harder ones. Include real user questions that caused problems.
- **Measure latency too:** 95% accurate in 30 seconds is unusable. 85% accurate in 2 seconds might be better for the use case.

Part 7: Observability and Monitoring

Seeing what your agent does in production. Tracing, logging, and diagnosing issues.

inmind.academy
by inmind.ai

Observability for LLM Applications

- User query (input)
- Retrieved chunks + similarity scores
- Full prompt sent to LLM
- LLM response (raw output)
- Tool calls (name, args, results)
- Tokens used (input + output)
- Latency at each step
- Cost estimate
- Error states

Tool	Type	Cost
LangSmith	Cloud (LangChain)	Free tier + paid
Phoenix (Arize)	Open source	Free
Langfuse	Open source	Free (self-host)
Weights & Biases	Cloud	Free tier
OpenLLMetry	Open source	Free

Minimum viable observability: log prompts and responses to a JSON file. Even simple logging is 100x better than nothing. You will need it the first time something goes wrong.

LangSmith: Tracing Your Agents

```
# Just set env vars - tracing is automatic
import os
os.environ["LANGCHAIN_TRACING_V2"] = "true"
os.environ["LANGCHAIN_API_KEY"] = "your-key"
os.environ["LANGCHAIN_PROJECT"] = "technova"

# Every invoke() is now traced!
```

LangSmith shows: full execution trace, input/output at each step, token counts, latency, tool calls, and error traces. Go to smith.langchain.com.

```
# What LangSmith shows:
[supervisor] -> "rag_agent" (32ms, 45tok)
  [rag_agent]
    [search_company_docs]
      query: "Pro plan price"
      result: "$470/year..."
  [rag_agent] -> answer (850ms, 120tok)
[supervisor] -> "math_agent" (28ms)
  [math_agent]
    [calculator]
      expr: "470 * 0.7"
      result: "329.00"
  [math_agent] -> answer (650ms, 85tok)
[supervisor] -> "FINISH" (25ms)

Total: 1.58s, 323 tokens
```

What's Next: Session 12 (Final Technical Session)

Today we added voice and vision to your agents and learned to properly evaluate your RAG systems.

Session 12 covers: Cloud AI platforms (AWS Bedrock, Azure AI, GCP Vertex), managed RAG services, inference optimization, hardware basics, security considerations, and the project architecture workshop.

.

After Session 12: DevOps/Docker session, then the final project.

inmind.academy
by inmind.ai

Voice Agent + Evaluation

Part A: Voice Pipeline (40%)

- Add voice input (Whisper) to your agent
- Add voice output (gTTS or better) to your agent
- Demo a voice conversation (3+ exchanges)
- Works in terminal or notebook

Part B: Evaluation (50%)

- Create test set: 15+ questions with ground truth
- Compute retrieval metrics (P@K, R@K, MRR)
- Evaluate generation (manual or RAGAS)
- Report metrics in a summary table
- Identify 3 failure cases with analysis

Bonus (10%)

- Compare 2 configurations and show which is better with data (chunk size, K, model, etc.)

inmind.academy
by inmind.ai

THANK YOU

Questions?

ADDRESS

Mkalles, main road, bld. 757, Lebanon

CONTACT

Phone +961 70 478 758
Email inmind.academy@inmind.ai
Website www.inmind.academy

SOCIAL MEDIA